

4.1 ПАРАЛЛЕЛЬНАЯ ПЕРЕДАЧА ДАННЫХ

Параллельная передача данных между контроллером и ВУ является по своей организации наиболее простым способом обмена. Для организации параллельной передачи данных помимо шины данных, количество линий в которой равно числу одновременно передаваемых битов данных, используется минимальное количество управляющих сигналов.

В простом контроллере внешнего устройства, обеспечивающем побайтную передачу данных на внешнее устройство (рис. 42), в шине связи с внешним устройством используются всего два управляющих сигнала: "Выходные данные готовы" и "Данные приняты".

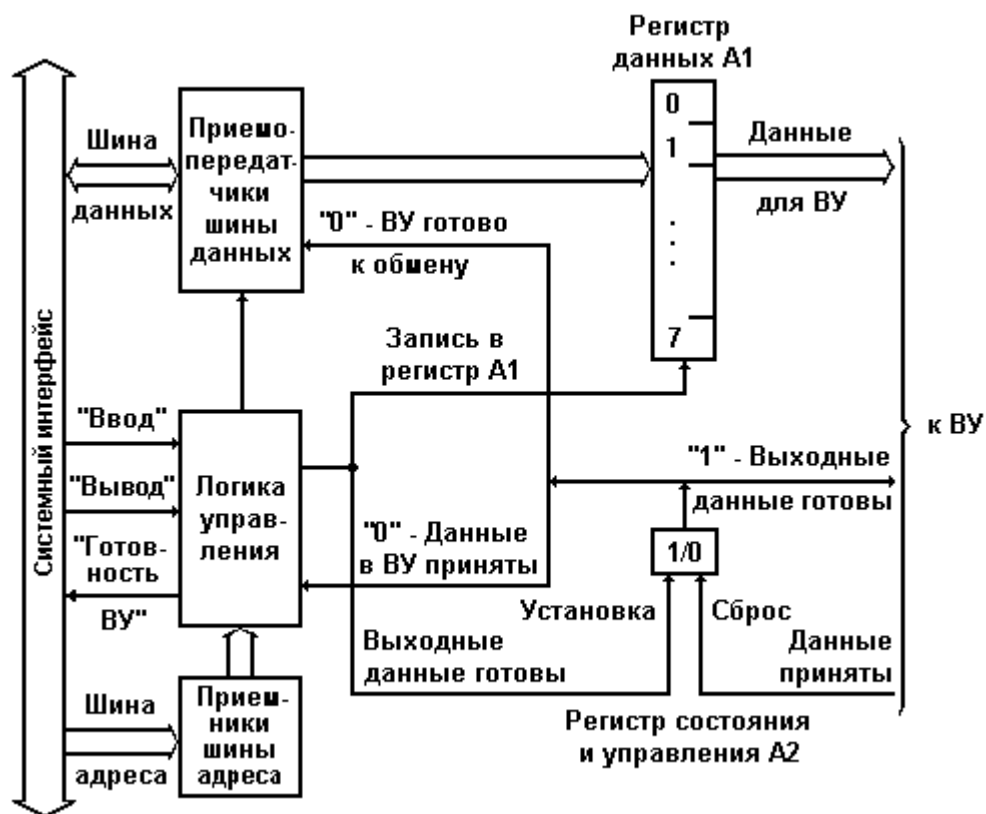


Рис. 42. Простой параллельный контроллер вывода.

Для формирования управляющего сигнала "Выходные данные готовы" и приема с внешнего устройства управляющего сигнала "Данные приняты" в контроллере используется одноразрядный адресуемый регистр состояния и управления A2 (обычно используются отдельные регистр состояния и регистр управления). Одновременно с записью очередного байта данных с

шины данных системного интерфейса в адресуемый регистр данных контроллера (порт вывода A1) в регистр состояния и управления записывается логическая единица. Тем самым формируется управляющий сигнал "Выходные данные готовы" в шине связи с внешним устройством.

Внешнее устройство, приняв байт данных, управляющим сигналом "Данные приняты" обнуляет регистр состояния контроллера. При этом формируются управляющий сигнал системного интерфейса "Готовность внешнего устройства" и признак готовности внешнего устройства к обмену, передаваемый в процессор по одной из линий шины данных системного интерфейса посредством стандартной операции ввода при реализации программы асинхронного обмена.

Логика управления контроллера обеспечивает селекцию адресов регистров контроллера, прием управляющих сигналов системного интерфейса и формирование на их основе внутренних управляющих сигналов контроллера, формирование управляющего сигнала системного интерфейса "Готовность внешнего устройства". Для сопряжения регистров контроллера с шинами адреса и данных системного интерфейса в контроллере используются соответственно приемники шины адреса и приемопередатчики шины данных.

Рассмотрим на примере, каким образом контроллер внешнего устройства обеспечивает параллельную передачу данных на внешнее устройство под управлением программы асинхронного обмена. Алгоритм асинхронного обмена в данном случае передачи прост

1. Микропроцессор проверяет готовность внешнего устройства к приему данных
2. Если внешнее устройство готово к приему данных (в данном случае это логический 0 в нулевом разряде регистра A2), то данные передаются с шины данных системного интерфейса в регистр данных A1 контроллера и далее на внешнее устройство. Иначе повторяется п. 1.

Блок-схема простого контроллера внешнего устройства, обеспечивающего побайтный прием данных с внешнего устройства, приведена на рис.43. В этом контроллере при взаимодействии с внешним устройством также используются два управляющих сигнала: "Данные от внешнего устройства готовы" и "Данные приняты".

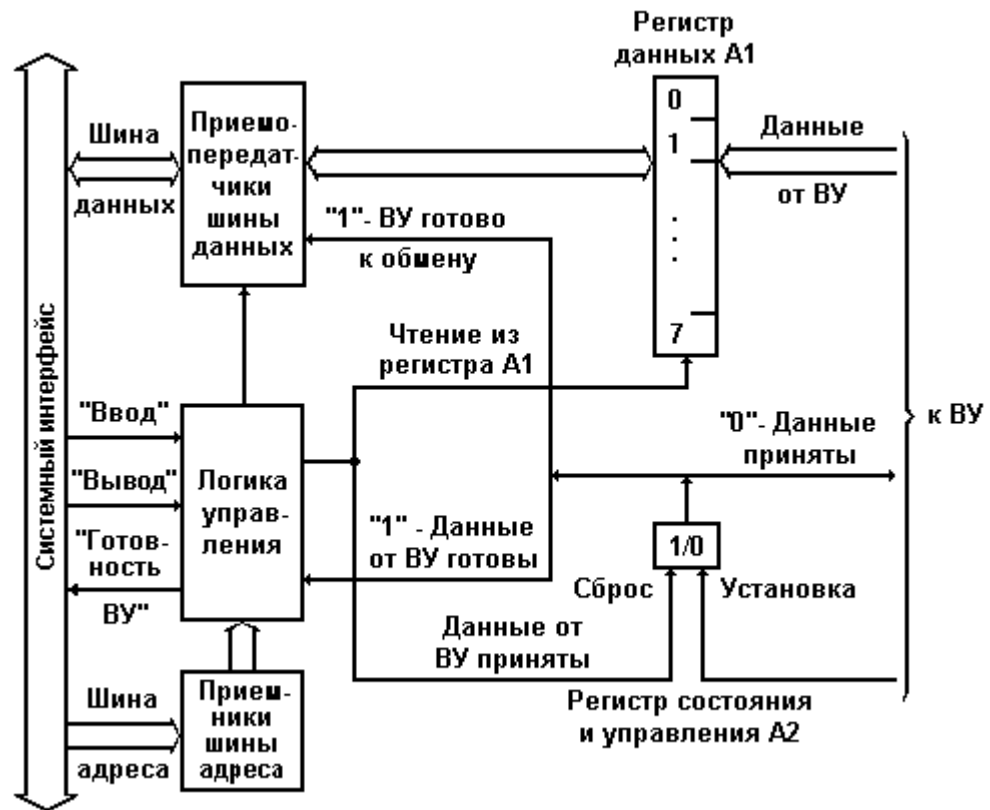


Рис. 43. Простой параллельный контроллер ввода

Для формирования управляющего сигнала "Данные приняты" и приема с внешнего устройства управляющего сигнала "Данные от внешнего устройства готовы" используется одноразрядный адресуемый регистр состояния и управления A2.

Внешнее устройство записывает в регистр данных контроллера A1 очередной байт данных и управляющим сигналом "Данные от внешнего устройства готовы" устанавливает в единицу регистр состояния и управления A2.

При этом формируются: управляющий сигнал системного интерфейса "Готовность внешнего устройства"; признак готовности внешнего

устройства к обмену, передаваемый в процессор по одной из линий шины данных системного интерфейса посредством операции ввода при реализации программы асинхронного обмена.

Тем самым контроллер извещает процессор о готовности данных в регистре A1. Процессор, выполняя программу асинхронного обмена, читает байт данных из регистра данных контроллера и обнуляет регистр состояния и управления A2. При этом формируется управляющий сигнал "Данные приняты" в шине связи с внешним устройством.

Логика управления контроллера и приемопередатчики шин системного интерфейса выполняют те же функции, что и в контроллере вывода.

Рассмотрим работу параллельного интерфейса ввода при реализации программы асинхронного обмена. Алгоритм асинхронного ввода так же прост, как и асинхронного вывода

1. Процессор проверяет наличие данных в регистре данных контроллера A1.

2. Если данные готовы (логическая 1 в регистре A2), то они передаются из регистра данных A1 на шину данных системного интерфейса и далее в регистр процессора или ячейку памяти микрокомпьютера. Иначе повторяется п. 1.

При параллельной передаче обеспечивается довольно высокая скорость обмена данными, которая ограничивается только быстродействием самого внешнего устройства.

4.2 ПОСЛЕДОВАТЕЛЬНАЯ ПЕРЕДАЧА ДАННЫХ

Использование последовательных линий связи для обмена данными с внешними устройствами возлагает на контроллеры внешнего устройства дополнительные по сравнению с контроллерами для параллельного обмена функции.

Во-первых, возникает необходимость преобразования формата данных из параллельного в последовательный при передаче информации на

внешнее устройство и преобразование из последовательного в параллельный формат при приеме данных с внешнего устройства.

Во-вторых, требуется реализовать соответствующий режиму работы внешнего устройства способ обмена данными: синхронный или асинхронный.

СИНХРОННЫЙ ПОСЛЕДОВАТЕЛЬНЫЙ ИНТЕРФЕЙС

Простой контроллер для синхронной передачи данных на внешнее устройство по последовательной линии связи (последовательный интерфейс) представлен на рис. 44.

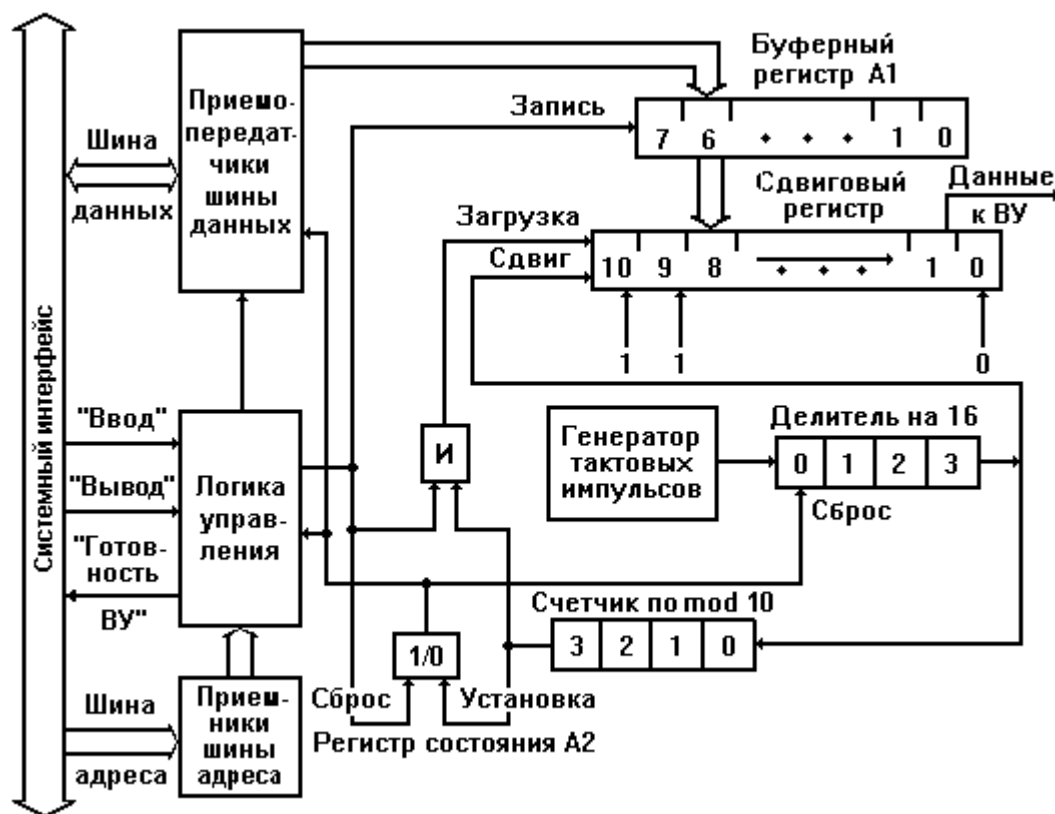


Рис. 44. Контроллер последовательной синхронной передачи

Восьмиразрядный адресуемый буферный регистр контроллера A1 служит для временного хранения байта данных до его загрузки в сдвиговый регистр. Запись байта данных в буферный регистр с шины данных системного интерфейса производится так же, как и в параллельном

интерфейсе, только при наличии единицы в одноразрядном адресуемом регистре состояния контроллера A2. Единица в регистре состояния указывает на готовность контроллера принять очередной байт в буферный регистр. Содержимое регистра A2 передается в процессор по одной из линий шины данных системного интерфейса и используется для формирования управляющего сигнала системного интерфейса "Готовность внешнего устройства". При записи очередного байта в буферный регистр A1 обнуляется регистр состояния A2.

Преобразование данных из параллельного формата, в котором они поступили в буферный регистр контроллера из системного интерфейса, в последовательный и передача их на линию связи производятся в сдвиговом регистре с помощью генератора тактовых импульсов и двоичного трехразрядного счетчика импульсов следующим образом.

Последовательная линия связи контроллера с внешним устройством подключается к выходу младшего разряда сдвигового регистра. По очередному тактовому импульсу содержимое сдвигового регистра сдвигается на один разряд вправо и в линию связи "Данные" выдается значение очередного разряда. Одновременно со сдвигом на внешнее устройство передается по отдельной линии "Синхронизация" тактовый импульс. Таким образом, каждый передаваемый по линии "Данные" бит информации сопровождается синхронизирующим сигналом по линии "Синхронизация", что обеспечивает его однозначное восприятие на приемном конце последовательной линии связи.

Количество переданных в линию тактовых сигналов, а следовательно, и переданных бит информации подсчитывается счетчиком тактовых импульсов. Как только содержимое счетчика становится равным 7, т. е. в линию переданы 8 бит (1 байт) информации, формируется управляющий сигнал "Загрузка", обеспечивающий запись в сдвиговый регистр очередного байта из буферного регистра. Этим же управляющим сигналом устанавливается в "1" регистр состояния. Очередным тактовым импульсом

счетчик будет сброшен в "0", и начнется очередной цикл выдачи восьми битов информации из сдвигового регистра в линию связи.

Синхронная последовательная передача отдельных битов данных на линию связи должна производиться без какого-либо перерыва, и следующий байт данных должен быть загружен в буферный регистр из системного интерфейса за время, не превышающее времени передачи восьми битов в последовательную линию связи.

При записи байта данных в буферный регистр обнуляется регистр состояния контроллера. Нуль в этом регистре указывает, что в линию связи передается байт данных из сдвигового регистра, а следующий передаваемый байт данных загружен в сдвиговый регистр.

Контроллер для последовательного синхронного приема данных из ВУ состоит из тех же компонентов, что и контроллер для синхронной последовательной передачи, за исключением генератора тактовых импульсов.

АСИНХРОННЫЙ ПОСЛЕДОВАТЕЛЬНЫЙ ИНТЕРФЕЙС

Организация асинхронного последовательного обмена данными с внешним устройством осложняется тем, что на передающей и приемной стороне последовательной линии связи используются настроенные на одну частоту, но физически разные генераторы тактовых импульсов и, следовательно, общая синхронизация отсутствует. Рассмотрим на примерах организацию контроллеров последовательных интерфейсов для последовательных асинхронных передачи и приема данных.

Простейший контроллер для асинхронной передачи данных на внешнее устройство по последовательной линии связи представлен на рис. 45. Он предназначен для передачи данных в формате с двумя стоповыми битами.

После передачи очередного байта данных в регистр состояния A2 записывается 1. Единичный выходной сигнал регистра A2 информирует

процессор о готовности контроллера к приему следующего байта данных и передаче его по линии связи на внешнее устройство. Этот же сигнал запрещает формирование импульсов со схемы выработки импульсов сдвига - делителя частоты сигналов тактового генератора на 16. Счетчик импульсов сдвига (счетчик по mod 10) находится в нулевом состоянии и его единичный выходной сигнал поступает на вентиль И, подготавливая цепь выработки сигнала загрузки сдвигового регистра.

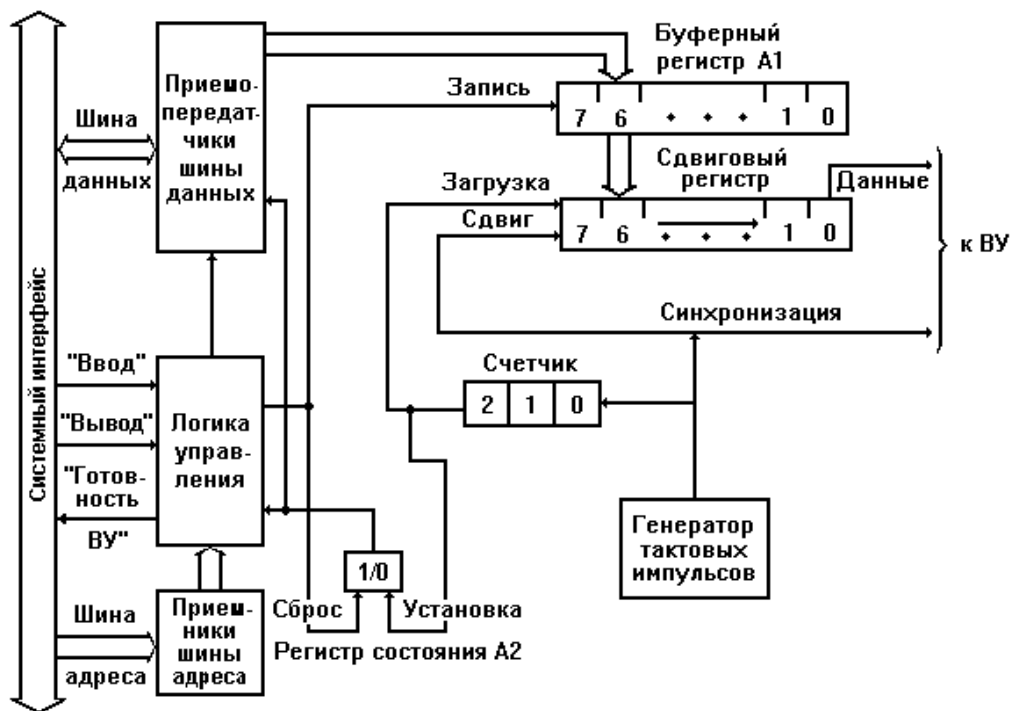


Рис. 45. Контроллер последовательной асинхронной передачи

Процесс передачи байта данных начинается с того, что процессор, выполняя команду "Выход", выставляет этот байт на шине данных. Одновременно процессор формирует управляющий сигнал системного интерфейса "Выход", по которому производятся запись передаваемого байта в буферный регистр А1, сброс регистра состояния А2 и формирование на вентиле И сигнала "Загрузка". Передаваемый байт переписывается в разряды 1, ..., 8 сдвигового регистра, в нулевой разряд сдвигового регистра записывается 0 (стартовый бит), а в разряды 9 и 10 - 1 (стоповые биты). Кроме того, снимается сигнал "Сброс" с делителя частоты, он начинает

Уровень логической единицы поступает по линии "Данные" в контроллер для асинхронного приема данных (рис. 46). Этот уровень создает условия для выработки сигнала, запрещающего работу делителя частоты генератора тактовых импульсов. Действительно, после приема предыдущего байта данных счетчик импульсов сдвига (счетчик по mod 9) находится в нулевом состоянии и на вентиль И поступают два единичных сигнала: со счетчика сдвигов и из линии "Данные". На выходе вентилей И вырабатывается сигнал сброса делителя частоты сигналов тактового генератора, запрещающий формирование импульсов сдвига.

В момент смены стопового бита на стартовый бит (момент начала передачи нового кадра) на линии "Данные" появится уровень логического нуля и тем самым будет снят сигнал сброса с делителя частоты. Состояние 4-разрядного двоичного счетчика (делителя частоты) начнет изменяться. Когда на счетчике накопится значение 8, он выдаст сигнал, поступающий на входы сдвигового регистра и счетчика импульсов сдвига. Так как частота сигналов генератора тактовых импульсов приемника должна совпадать с частотой генератора тактовых импульсов передатчика, то сдвиг (считывание) бита произойдет примерно на середине временного интервала, отведенного на передачу бита данных, т. е. времени, необходимого для выработки шестнадцати тактовых импульсов. Это делается для уменьшения вероятности ошибки из-за возможного различия частот генераторов передатчика и приемника, искажения формы передаваемых сигналов (переходные процессы) и т. п. Следующий сдвиг произойдет после прохождения шестнадцати тактовых импульсов, т. е. на середине временного интервала передачи первого информационного бита.

При приеме в сдвиговый регистр девятого бита кадра (восьмого информационного бита) из него "выдвинется" стартовый бит и, следовательно, в сдвиговом регистре будет размещен весь принятый байт информации. В этот момент счетчик импульсов сдвига придет в нулевое состояние и на его выходе будет выработан единичный сигнал, по которому

содержимое сдвигового регистра переписывается в буферный регистр, в регистр состояния A2 записывается 1 и он будет информировать процессор об окончании приема очередного байта, вентиль И подготовится к выработке сигнала "Сброс" (этот сигнал сформируется после прихода первого стопового бита).

Получив сигнал готовности (1 в регистре A2), процессор выполнит команду "Ввод". При этом вырабатывается управляющий сигнал системного интерфейса "Ввод", по которому производится пересылка принятого байта данных из буферного регистра в процессор (сигнал "Чтение") и сброс регистра состояния A2.

1. СПОСОБЫ ОБМЕНА ИНФОРМАЦИЕЙ В МИКРОПРОЦЕССОРНОЙ СИСТЕМЕ

Применяются три режима ввода/вывода:

- программно-управляемый ввод-вывод информации (называемый также программным или нефорсированным вводом-выводом),
- ввод-вывод информации по прерываниям (форсированный ввод-вывод)
- прямой доступ к памяти.

Первый из них характеризуется тем, что инициирование и управление ввода-вывода информации осуществляется программой, выполняемой процессором, а внешние устройства играют сравнительно пассивную роль и сигнализируют только о своем состоянии, в частности, о готовности к операциям ввода/вывода. Во втором режиме ввод-вывод информации иницируется не процессором, а внешним устройством, генерирующим специальный сигнал прерывания. Реагируя на этот сигнал готовности устройства к передаче данных, процессор передает управление подпрограмме обслуживания устройства, вызвавшего прерывание. Действия, выполняемые этой подпрограммой, определяются пользователем, а непосредственными операциями вводом-выводом информации управляет

процессор. Наконец, в режиме прямого доступа к памяти, который используется, когда пропускной способности процессора недостаточно, действия процессора приостанавливаются, он отключается от системной шины и не участвует в передачах данных между основной памятью и быстродействующим внешним устройством. Заметим, что во всех вышеуказанных случаях основные действия, выполняемые системной шиной, подчиняются двум основным принципам.

1. В процессе взаимодействия любых двух электронных устройств одно из них обязательно выполняет активную, управляющую роль и является задатчиком, второе оказывается управляемым, исполнителем. Чаще всего задатчиком является процессор.

2. Другим важным принципом, заложенным в структуру интерфейса, является принцип квитирования (запроса - ответа): каждый управляющий сигнал, посланный задатчиком, подтверждается сигналом исполнителя. При отсутствии ответного сигнала исполнителя в течение заданного интервала времени формируется так называемый тайм-аут, задатчик фиксирует ошибку обмена и прекращает данную операцию.

Программно-управляемый ввод/вывод

Данный режим характеризуется тем, что все действия по вводу/выводу реализуются командами прикладной программы. Наиболее простыми эти действия оказываются для "всегда готовых" внешних устройств, например индикатора на светодиодах. При необходимости ввод-вывод информации в соответствующем месте программы используются команды IN или OUT. Такая передача данных называется синхронным или безусловным вводом-выводом информации.

Однако для большинства внешних устройств до выполнения операций ввод-вывод информации надо убедиться в их готовности к обмену, т.е. ввод-вывод информации является асинхронным. Общее состояние устройства характеризуется флагом готовности READY, называемым также флагом готовности/занятости (READY/BUSY). Иногда состояния готовности и

занятости идентифицируются отдельными флагами READY и BUSY, входящими в слово состояния устройства.

Процессор проверяет флаг готовности с помощью одной или нескольких команд. Если флаг установлен, то инициируются собственно ввод или вывод одного или нескольких слов данных. Когда же флаг сброшен, процессор выполняет цикл из 2-3 команд с повторной проверкой флага READY до тех пор, пока устройство не будет готово к операциям ввода-вывода (рис. 47). Данный цикл называется циклом ожидания готовности ВУ и реализуется в различных процессорах по-разному.



Рис. 47. Цикл программного ожидания готовности внешнего устройства

Основной недостаток программного ввода-вывода связан с непроизводительными потерями времени процессора в циклах ожидания. К достоинствам следует отнести простоту его реализации, не требующей дополнительных аппаратных средств.

Организация прерываний

Одной из разновидностей программно-управляемого обмена данными с внешнего устройства в микропроцессорную систему является обмен с прерыванием программы, отличающийся от асинхронного программно-управляемого обмена тем, что переход к выполнению команд, физически реализующих обмен данными, осуществляется с помощью специальных аппаратных средств. Команды обмена данными в этом случае выделяют в

отдельный программный модуль - подпрограмму обработки прерывания. Задачей аппаратных средств обработки прерывания в процессоре как раз и является приостановка выполнения одной программы (ее еще называют основной программой) и передача управления подпрограмме обработки прерывания. Действия, выполняемые при этом процессором, как правило, те же, что и при обращении к подпрограмме. Только при обращении к подпрограмме они инициируются командой, а при обработке прерывания - управляющим сигналом от внешнего устройства, который называют "Запрос на прерывание" или "Требование прерывания".

Эта важная особенность обмена с прерыванием программы позволяет организовать обмен данными с внешнего устройства в произвольные моменты времени, не зависящие от программы, выполняемой в микропроцессорной системе. Таким образом, появляется возможность обмена данными с внешнего устройства в реальном масштабе времени, определяемом внешней по отношению к микропроцессорной системе средой. Обмен с прерыванием программы существенным образом экономит время процессора, затрачиваемое на обмен. Это происходит за счет того, что исчезает необходимость в организации программных циклов ожидания готовности внешнего устройства, на выполнение которых тратится значительное время, особенно при обмене с медленными внешними устройствами.

Прерывание программы по требованию внешнего устройства не должно оказывать на прерванную программу никакого влияния кроме увеличения времени ее выполнения за счет приостановки на время выполнения подпрограммы обработки прерывания. Поскольку для выполнения подпрограммы обработки прерывания используются различные регистры процессора (счетчик команд, регистр состояния и т.д.), то информацию, содержащуюся в них в момент прерывания, необходимо сохранить для последующего возврата в прерванную программу.

Обычно задача сохранения содержимого счетчика команд и регистра состояния процессора возлагается на аппаратные средства обработки прерывания. Сохранение содержимого других регистров процессора, используемых в подпрограмме обработки прерывания, производится непосредственно в подпрограмме. Отсюда следует достаточно очевидный факт: чем больший объем информации о прерванной программе сохраняется программным путем, тем больше время реакции микропроцессорной системы на сигнал прерывания, и наоборот. Предпочтительными с точки зрения повышения производительности являются уменьшение числа команд, обеспечивающих сохранение информации о прерванной программе, и реализация этих функций аппаратными средствами.

Формирование сигналов прерываний - запросов внешнего устройства на обслуживание происходит в контроллерах соответствующих внешних устройств. В простейших случаях в качестве сигнала прерывания может использоваться сигнал "Готовность внешнего устройства", поступающий из контроллера внешнего устройства в системный интерфейс. Однако такое решение обладает существенным недостатком - процессор не имеет возможности управлять прерываниями, т. е. разрешать или запрещать их для отдельных внешних устройств. В результате организация обмена данными в режиме прерывания с несколькими внешними устройствами существенно усложняется.

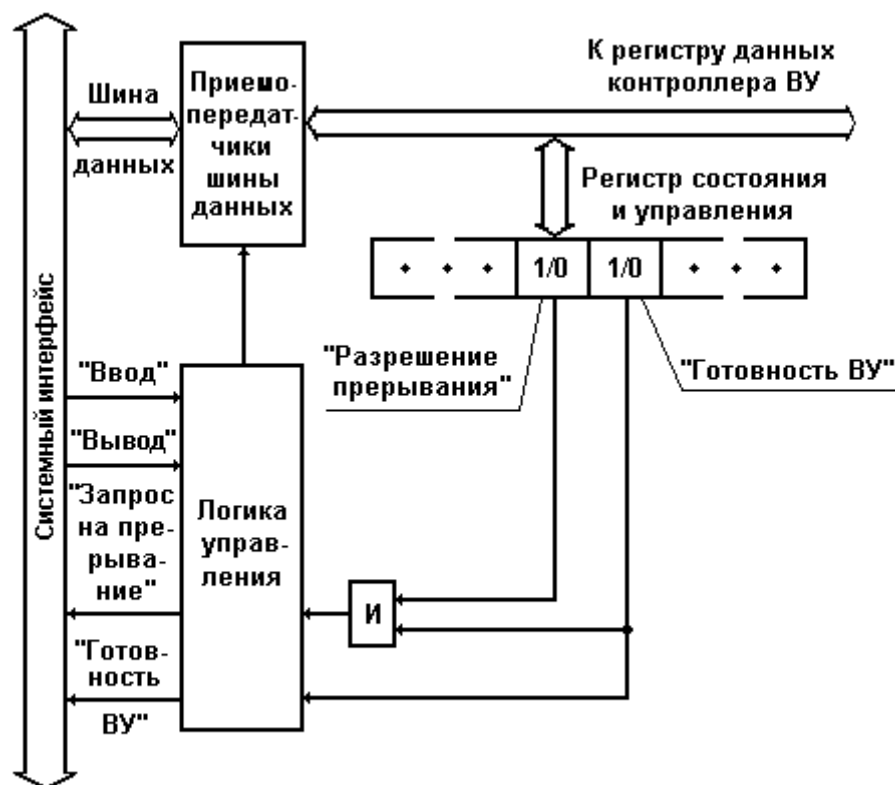


Рис. 48. Фрагмент блок-схемы контроллера внешнего устройства с разрядом "Разрешение прерывания" в регистре состояния и управления

Для решения этой проблемы регистр состояния и управления контроллера внешнего устройства (рис. 48) дополняют еще одним разрядом - "Разрешение прерывания". Запись 1 или 0 в разряд "Разрешение прерывания" производится программным путем по одной из линий шины данных системного интерфейса. Управляющий сигнал системного интерфейса "Запрос на прерывание" формируется с помощью схемы совпадения только при наличии единиц в разрядах "Готовность ВУ" и "Разрешение прерывания" регистра состояния и управления контроллера.

Обычно используется одноуровневая система прерываний, т. е. сигналы "Запрос на прерывание" от всех внешних устройств поступают на один вход процессора. Поэтому возникает проблема идентификации внешнего устройства, запросившего обслуживание, и реализации заданной очередности (приоритета) обслуживания внешних устройств при одновременном поступлении нескольких сигналов прерывания. Существуют

два основных способа идентификации внешних устройств, запросивших обслуживания:

- программный опрос регистров состояния (разряд "Готовность внешнего устройства") контроллеров всех внешних устройств;
- использование векторов прерывания.

Организация прерываний с программным опросом готовности предполагает наличие в памяти микропроцессорной системы единой подпрограммы обслуживания прерываний от всех внешних устройств. Структура такой подпрограммы приведена на рис. 49.

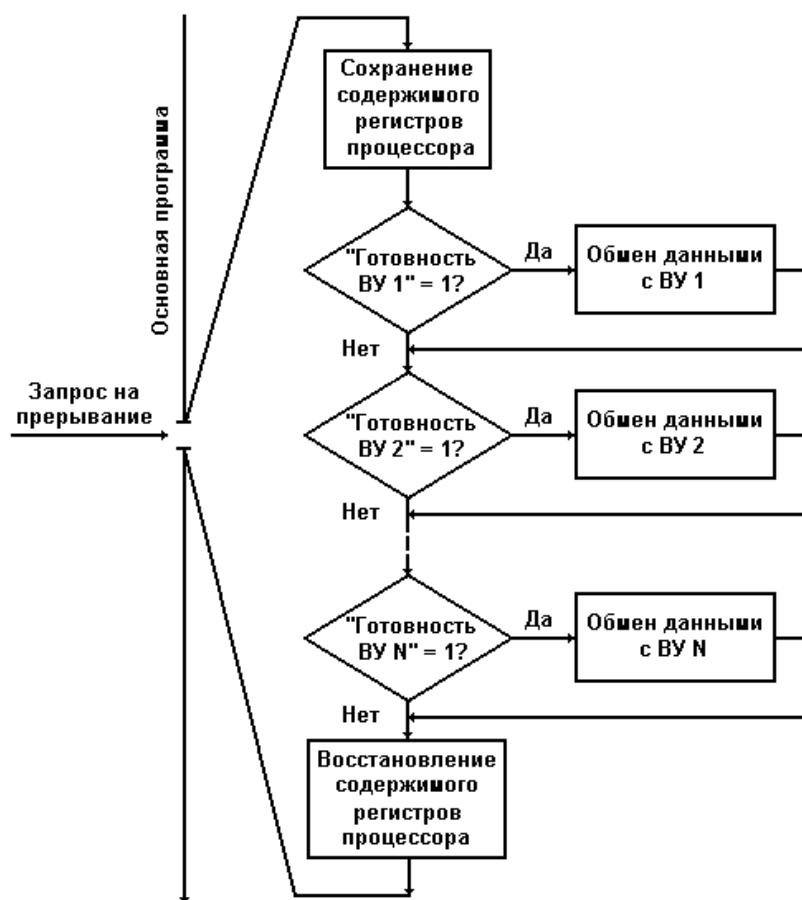


Рис. 49. Структура единой программы обработки прерываний и ее связь с основной программой

Обслуживание внешнего устройства с помощью единой подпрограммы обработки прерываний производится следующим образом. В конце последнего машинного цикла выполнения очередной команды основной программы процессор проверяет наличие требования прерывания от

внешнего устройства. Если сигнал прерывания есть и в процессоре прерывание разрешено, то процессор переключается на выполнение подпрограммы обработки прерываний.

После сохранения содержимого регистров процессора, используемых в подпрограмме, начинается последовательный опрос регистров состояния контроллеров всех внешних устройств, работающих в режиме прерывания. Как только подпрограмма обнаружит готовое к обмену внешнее устройство, сразу выполняются действия по его обслуживанию. Завершается подпрограмма обработки прерывания после опроса готовности всех внешних устройств и восстановления содержимого регистров процессора.

Приоритет внешнего устройства в микропроцессорной системе с программным опросом готовности внешнего устройства однозначно определяется порядком их опроса в подпрограмме обработки прерываний. Чем раньше в подпрограмме опрашивается готовность внешнего устройства, тем меньше время реакции на его запрос и выше приоритет. Необходимость проверки готовности всех внешних устройств существенно увеличивает время обслуживания тех внешних устройств, которые опрашиваются последними. Это является основным недостатком рассматриваемого способа организации прерываний. Поэтому обслуживание прерываний с опросом готовности внешнего устройства используется только в тех случаях, когда отсутствуют жесткие требования на время обработки сигналов прерывания внешних устройств.

Организация системы прерываний с использованием векторов прерываний позволяет устранить указанный недостаток. При такой организации системы прерываний внешних устройств, запросившее обслуживания, само идентифицирует себя с помощью вектора прерывания - адреса ячейки основной памяти микропроцессорной системы, в которой хранится либо первая команда подпрограммы обслуживания прерывания данного внешнего устройства, либо адрес начала такой подпрограммы. Таким образом, процессор, получив вектор прерывания, сразу переключается

на выполнение требуемой подпрограммы обработки прерывания. В микропроцессорной системе с векторной системой прерывания каждое внешнее устройство должно иметь собственную подпрограмму обработки прерывания.

Различают векторные системы с интерфейсным и внеинтерфейсным вектором. В первом случае вектор прерывания (или его адрес) формирует контроллер внешнего устройства, запросившего обслуживания, во втором - контроллер прерываний, общий для всех устройств, работающих в режиме прерываний (IBM-совместимые персональные компьютеры).

Рассмотрим организацию векторной системы с интерфейсным вектором. Вектор прерывания (или его адрес) выдается контроллером не одновременно с запросом на прерывание, а только по разрешению процессора, как это реализовано в схеме на рис. 50. Это делается для того, чтобы исключить одновременную выдачу векторов прерывания от нескольких внешних устройств. В ответ на сигнал контроллера внешнего устройства "Запрос на прерывание" процессор формирует управляющий сигнал "Предоставление прерывания (вх.)", который разрешает контроллеру внешнего устройства, запросившему обслуживание, выдачу вектора прерывания в шину адреса системного интерфейса. Для этого в контроллере используются регистр вектора прерывания и схема совпадения ИЗ. Регистр вектора прерывания обычно реализуется с помощью перемычек или переключателей, что позволяет пользователю устанавливать для конкретных внешних устройств требуемые значения векторов прерывания.

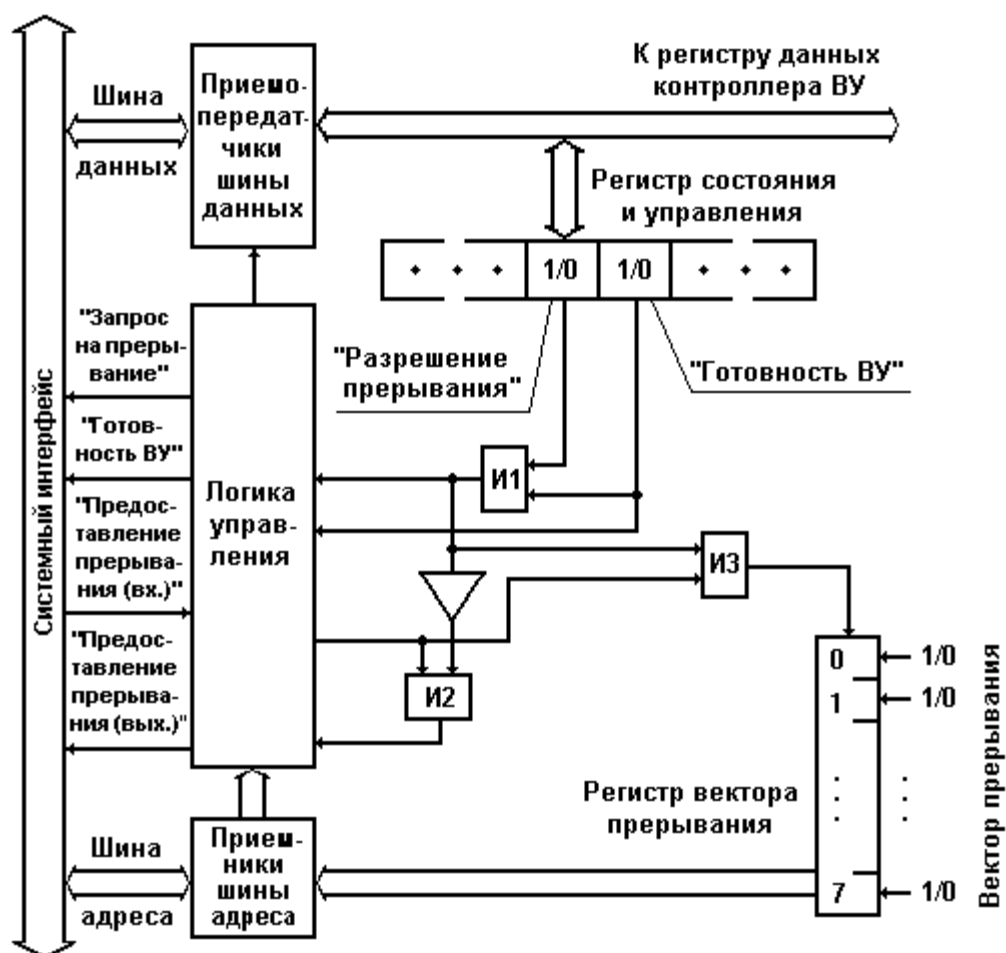


Рис. 50. Формирование векторов прерывания в контроллере ВУ

Управляющий сигнал "Предоставление прерывания (вых.)" формируется в контроллере ВУ с помощью схемы совпадения И2. Этот сигнал используется для организации последовательного аппаратного опроса готовности внешних устройств и реализации тем самым требуемых приоритетов внешних устройств. Процессор при поступлении в него по общей линии системного интерфейса "Запрос на прерывание" сигнала прерывания формирует управляющий сигнал "Предоставление прерывания (вх.)", который поступает сначала в контроллер внешнего устройства с наивысшим приоритетом (рис. 51). Если это устройство не требовало обслуживания, то его контроллер пропускает сигнал "Предоставление прерывания" на следующий контроллер, иначе дальнейшее распространение сигнала прекращается и контроллер выдает вектор прерывания на адресно-информационную шину.



Рис. 51. Реализация приоритетов ВУ в микроЭВМ с векторной системой прерываний с интерфейсным вектором (ППР (вх.) - "Предоставление прерывания (входной)"; "ППР (вых.) - Предоставление прерывания (выходной)")

Аппаратный опрос готовности внешних устройств производится гораздо быстрее, нежели программный. Но если обслуживания запросили одновременно два или более внешних устройства, обслуживание менее приоритетных внешних устройств будет отложено на время обслуживания более приоритетных, как и в системе прерывания с программным опросом.

Векторная система с внеинтерфейсным вектором прерывания используется в IBM-совместимых персональных компьютерах. В этих компьютерах контроллеры внешних устройств не имеют регистров для хранения векторов прерывания, а для идентификации устройств, запросивших обслуживания, используется общий для всех внешних устройств контроллер прерываний. Ниже приведен пример контроллера прерываний INTEL 8259A.

БИС программируемого контроллера прерываний (ПКП) представляет собой устройство, реализующее до восьми уровней запросов на прерывания с возможностью программного маскирования и изменения порядка обслуживания прерываний. За счет каскадного включения БИС ПКП число уровней прерывания может быть расширено до 64 (в архитектуре персонального компьютера IBM PC AT - 16).

Структурная схема ПКП приведена на рисунке 52.

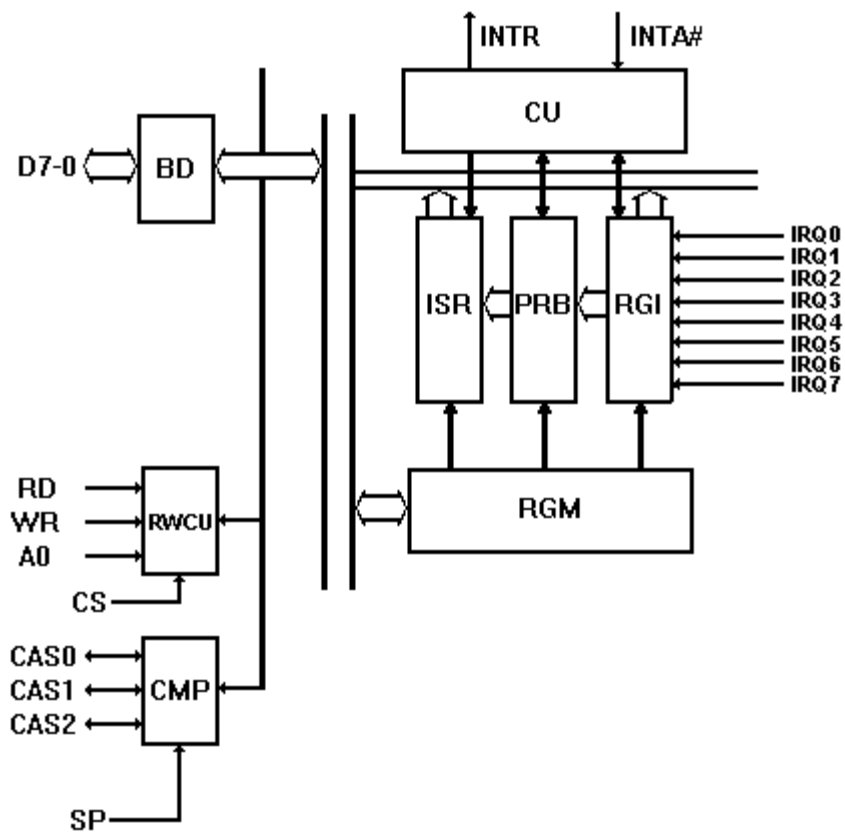


Рис. 52. Контроллер прерываний Intel 8259A

В состав БИС входят:

RGI - регистр запретов прерываний; хранит все уровни, на которые поступают запросы IRQx;

PRB - схема принятия решений по приоритетам; схема идентифицирует приоритет запросов и выбирает запрос с наивысшим приоритетом;

ISR - регистр обслуживаемых прерываний; сохраняет уровни запросов прерываний, находящиеся на обслуживании ПКП;

RGM - регистр маскирования прерываний; обеспечивает запрещение одной или нескольких линий запросов прерывания;

BD - буфер данных; предназначен для сопряжения ПКП с системной шиной данных;

RWCU - блок управления записью/чтением; принимает управляющие сигналы от микропроцессора и задает режим функционирования ПКП;

CMP - схема каскадного буфера-компаратора; используется для включения

в систему нескольких ПКП;

CU - схема управления; вырабатывает сигналы прерывания и формирует трехбайтовую команду CALL для выдачи на шину данных.

Установка ПКП в исходное состояние и "настройка" его на определенный режим обслуживания прерываний происходит с помощью двух типов команд: команд инициализации (ICW) и команд управления операциями (OCW).

Программируемый контроллер прерываний (ПКП) имеет 16 входов запросов прерываний (IRQ 0 - IRQ 15). Контроллер состоит из двух каскадно включенных контроллеров - выход INTR (запрос на прерывание) второго контроллера подключен ко входу IRQ 2 первого контроллера. В качестве примера отметим, что к линии IRQ 0 подключен системный таймер, к линии IRQ 1 - клавиатура, к линии IRQ 8 - часы реального времени и т.д.

Упрощенная схема взаимодействия контроллера прерываний с процессором и контроллером шины имеет следующий вид.

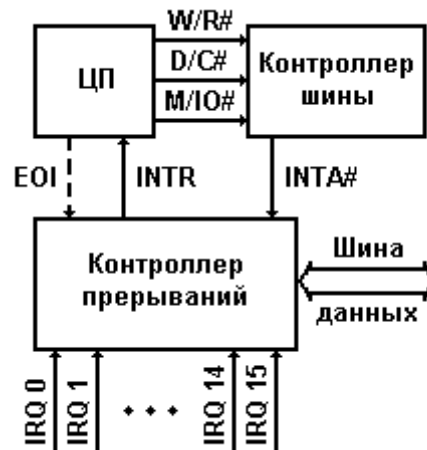


Рис. 53. Упрощенная схема взаимодействия контроллера прерываний с процессором и контроллером шины в IBM-совместимых персональных компьютерах класса AT

Эта схема функционирует следующим образом. Пусть в некоторый момент времени контроллер клавиатуры с помощью единичного сигнала по линии IRQ 1 известил контроллер прерываний о своей готовности к обмену. В ответ на запрос контроллер прерываний генерирует сигнал INTR (запрос на прерывание) и посылает его на соответствующий вход процессора.

Процессор, если маскируемые прерывания разрешены (т.е. установлен флаг прерываний IF в регистре флагов процессора), посылает на контроллер шины сигналы R# - чтение, C# - управление и IO# - ввод/вывод, определяющие тип цикла шины. Контроллер шины, в свою очередь, генерирует два сигнала подтверждения прерывания INTA# и направляет их на контроллер прерываний. По второму импульсу контроллер прерываний выставляет на шину данных восьмибитный номер вектора прерывания, соответствующий данной линии IRQ.

В режиме реального адреса ("реальном" режиме) векторы прерываний хранятся в таблице векторов прерываний, которая находится в первом килобайте оперативной памяти. Под каждый вектор отведено 4 байта (2 байта под адрес сегмента и 2 байта под смещение), т.е. в таблице может содержаться 256 векторов. Адрес вектора в таблице - номер вектора * 4.

Далее процессор считывает номер вектора прерывания. Сохраняет в стеке содержимое регистра флагов, сбрасывает флаг прерываний IF и помещает в стек адрес возврата в прерванную программу (регистры CS и IP). После этого процессор извлекает из таблицы векторов прерываний адрес подпрограммы обработки прерываний для данного устройства и приступает к ее выполнению.

Процедура обработки аппаратного прерывания должна завершаться командой конца прерывания EOI (End of Interruption), посылаемой контроллеру прерываний. Для этого необходимо записать байт 20h в порт 20h (для первого контроллера) и в порт A0h (для второго).

В IBM PC/XT/AT используется режим прерываний с фиксированными приоритетами. Высшим приоритетом обладает запрос по линии IRQ 0, низшим - IRQ 7. Так как второй контроллер подключен к линии IRQ 2 первого контроллера, то приоритеты линий IRQ в порядке убывания приоритета располагаются следующим образом: IRQ 0, IRQ 1, IRQ 8 - IRQ 15, IRQ 3 - IRQ 7. Если запрос на обслуживание посылают одновременно два устройства с разными приоритетами, то контроллер обслуживает запрос с

большим приоритетом, а запрос с меньшим приоритетом блокирует.
Блокировка сохраняется до получения команды EOI.